

Design Document



פ ת ט (“Patt”) Processor

Microarchitecture
ECE 382N.19
Dr. Yale Patt

Authors:

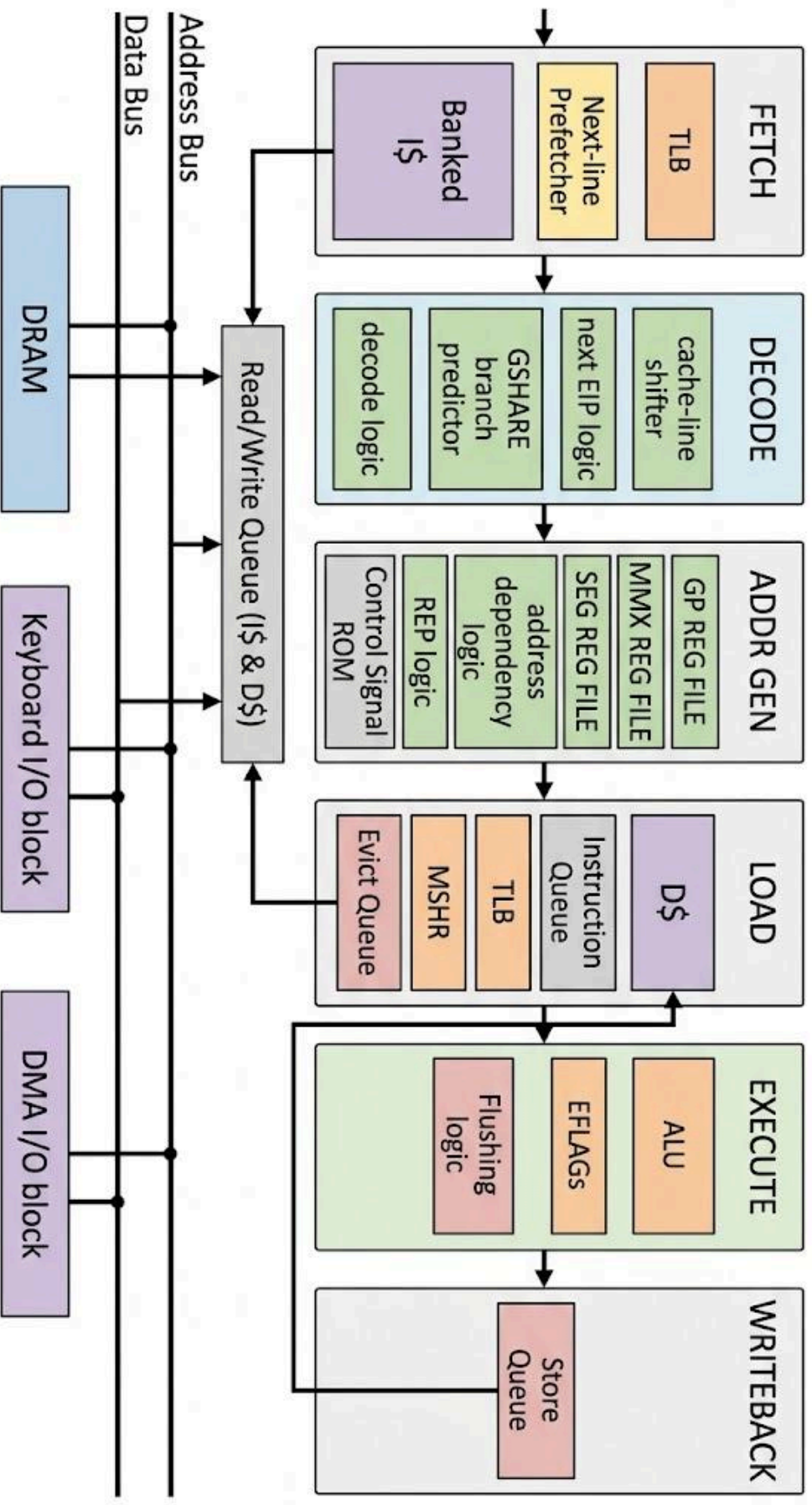
Anthony Hermez
Jiwoo Park
Roy Mor

Reviewers:

Dr. Yale Patt
Orhan Unuvar
Asher Nederveld
Edgar Turcotte

Acknowledgements

We would like to express our sincere gratitude to Dr. Yale Patt for his invaluable mentorship and for challenging us to reach a deeper understanding of microarchitecture. We are also deeply appreciative of each of our teaching assistants for their constant support throughout the semester: Orhan Unuvar for his technical insights and persistent availability; Asher Nederveld for his constructive feedback on our design choices; and Edgar Turcotte for his guidance through the complexities of the architecture. Their collective expertise was instrumental to the successful development of the $\mathfrak{P} \vdash \mathfrak{P}$ (“Patt”) Processor. Finally, we would like to thank our families and friends for their unwavering encouragement and patience as we dedicated ourselves to completing this project.



Fetch

The Fetch stage delivers two sequential 16-byte cache lines to Decode every cycle so that variable-length x86 instructions can cross cache-line boundaries without a bubble. It is built around a banked instruction cache, a three-line lookahead next-line prefetcher, a multi-port TLB, and a flushable FE/DE latch.

Banked Instruction Cache

The I-cache (`core/icache/banked_icache.v`) is a 2-bank, 4-way set-associative cache with 16-byte lines:

- **Total capacity:** $2 \text{ banks} \times 4 \text{ sets/bank} \times 4 \text{ ways/set} \times 16 \text{ B/line} = 512 \text{ bytes}$.
- **Indexing:** virtual-address bits [5:3] select one of four sets per bank; bit [4] selects the bank pair so that the second physical line lives in the opposite bank.
- **Tags:** 9 bits per way (4-bit PFN from the TLB concatenated with VA[7:3]).
- **Replacement:** per-set absolute LRU managed by `core/icache/lru_bank_icache.v`.

Each lookup drives both banks combinationally and produces a low-line / high-line pair via two 16-byte muxes selected by `eip[4]` (`FETCH.v` lines 185–209). The output also carries per-line valid bits (`LOW_valid`, `HIGH_valid`) and per-line GP/PF flags so Decode can refuse to decode bytes on faulting pages.

Next-Line Prefetcher

The next-line prefetcher (`core/prefetcher/nl_prefetcher.v`) maintains three outstanding lookahead targets `PA_0`, `PA_1`, `PA_2` corresponding to the cache lines containing `EIP`, `EIP + 16`, and `EIP + 32`. On every cycle each target is checked in parallel against the I-cache hit logic and against a 32-entry searchable FIFO that holds outstanding bus requests; only targets that miss the cache, are not already in flight, and are not blocked by a write-back are enqueued. The search FIFO supports $K = 3$ read ports and three write ports per cycle so all three lookahead lines can be enqueued together. There is no separate I-cache MSHR — the search FIFO performs the same merge function. When the bus arbiter grants a request, `BUS_REQ_TAG` and `BUS_REQ_IDX` drop out of the FIFO head and a single-cycle `BUS_W_EN` pulse from the I-cache arbiter writes the returning line into the chosen way.

Address Translation in Fetch

The I-cache is virtually indexed and physically tagged. The prefetcher forms each line's virtual tag by adding (`CS << 12`) to `EIP[31:4]` (28-bit segmented base computation in `nl_prefetcher.v` lines 260–290) and presents it to the TLB. Three of the TLB's nine lookup ports are dedicated to Fetch (ports 0–2); they translate the three lookahead VAs in parallel and drive PFNs into both the

cache's tag-comparison path and the prefetcher's outstanding-request enqueue logic. A page-fault on any line propagates as PF[i]; a non-cacheable page (PCD = 1) suppresses cache writes so that uncached / MMIO regions do not pollute the I-cache.

FE → DE Latches

The `decode_latch` (`core/fetch/decode_latch.v`) latches the two cache lines plus their per-line validity, per-line GP / PF flags, and the effective address of the upper line, into DFFs partitioned into four 16-byte slots so that downstream Decode can address any byte within the 32-byte window. On flush, the latch is cleared by anding `flush_reset_b` with the inverse of `exception_in_execute`. When EXECUTE signals an exception, the latch swaps the cached lines for a hard-coded exception dispatch pattern generated on-the-fly from the IDTR base (0x02000010) and the faulting EIP, so that Decode immediately observes a 13-byte software exception sequence on the cycle after the flush.

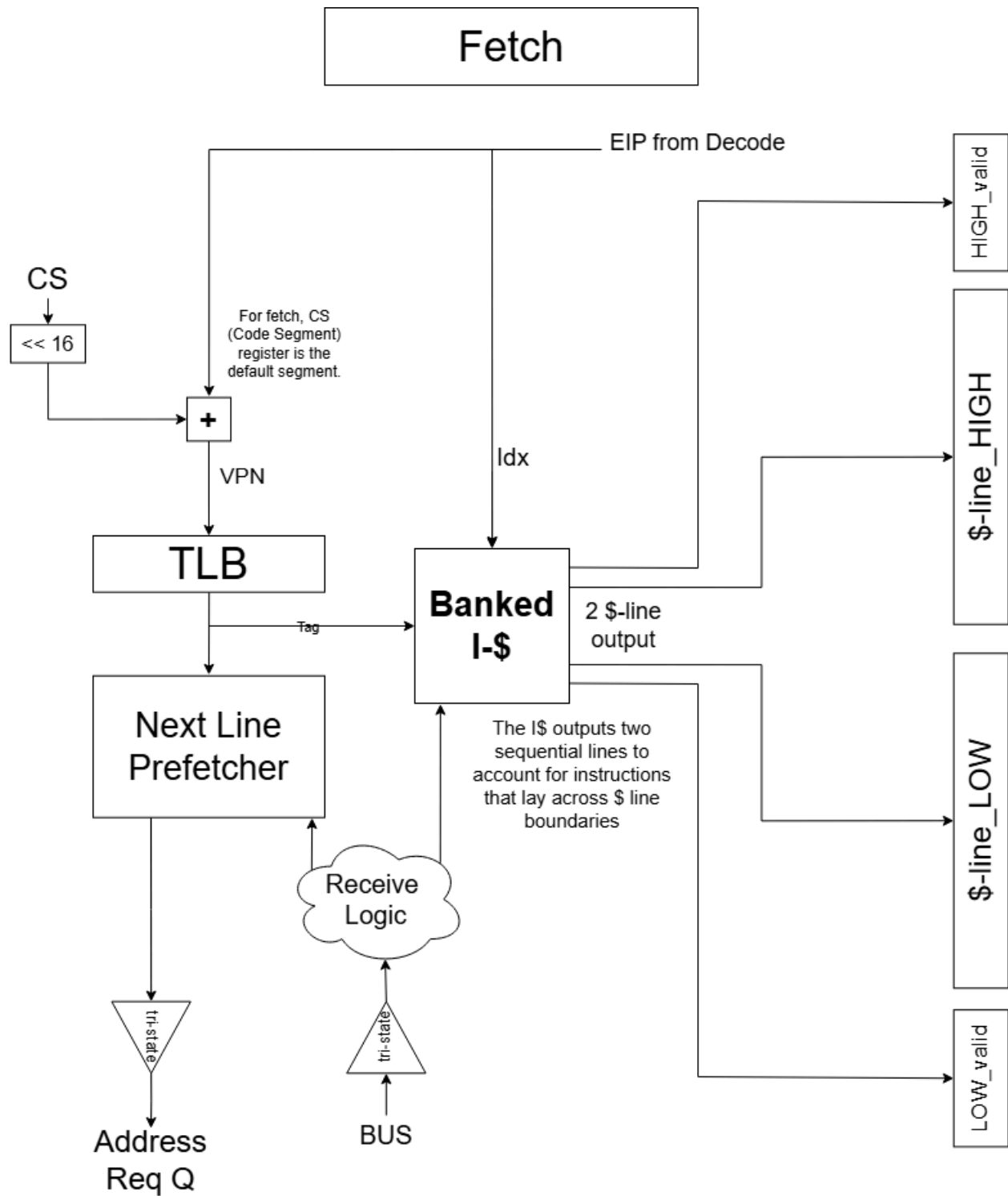


Figure 1. Fetch Stage Diagram.

Decode

Decode converts the 32-byte rolling window from Fetch into a fully decoded x86 instruction, computes its length, generates the next EIP, and produces a Branch Prediction. It is the deepest combinational stage in the pipeline and the only place a ROM is consulted for control bits.

Window Shifter

`decode_shifter` (`core/decode/decode_shifter.v`) is a parametric mux array that aligns the 32-byte cache-line pair down to a 15-byte (120-bit) potential-instruction window starting at EIP's cache offset. It is implemented as eleven cascaded `mux32_16_fast` multiplexers, each selecting a 16-bit chunk based on `eip[3:0]` plus an `eip-in-high-line` bit. Per-byte valid, GP, and PF bits are shifted by parallel mux arrays so Decode can mark the instruction invalid if any consumed byte sits on a fault-flagged line. The last byte position is zero-padded because the 368-bit shifted input cannot reach byte 15 at the maximum offset.

Length and Field Selection

Length decode is performed by the dedicated combinational block formed by `is_prefix`, `opcode_pos`, `opcode_sel`, `modrm_sel`, `sib_sel`, `disp_sel`, `disp_size_sel`, `imm_sel`, `imm_op_size`, and `inst_len`:

- **Prefix detection** (`is_prefix.v`): byte-position-aware comparators recognise eight prefixes — 0x66 (operand-size override), 0x2E / 0x36 / 0x3E / 0x26 (CS / SS / DS / ES segment override), 0x64 / 0x65 (FS / GS override), and 0xF3 (REP). Up to three contiguous prefix bytes are honoured.
- **Opcode position** (`opcode_pos.v`): a 2-bit count of leading prefixes, used to address every other field selector.
- **Two-byte opcode**: detected as 0x0F at the opcode position; `opcode_sel.v` then exposes the actual opcode byte to the decode ROM.
- **ModRM extraction** (`modrm_sel.v`): a 4-way mux over bytes B1...B5 selects ModRM, then a two-byte-opcode mux picks the correct alignment. Fields `{mode[7:6], reg[5:3], rm[2:0]}` are exposed.
- **SIB extraction** (`sib_sel.v`): an SIB byte is present only when `has_modrm = 1`, `modrm_rm == 3'b100`, and `modrm_mode != 2'b11`. Modes that skip SIB are register-direct (`mod = 11`), register-indirect with `rm != 4`, and forms without ModRM. The decode logic also short-circuits the `index = 100` (none) and `base = 101` (`disp32`) corner cases so that those SIB encodings never propagate effective-address terms.
- **Displacement size** (`disp_size_sel.v`): `mod = 00` → 0 bytes (except `mod = 00`, `rm = 101` → forced 4 bytes), `mod = 01` → 1 byte, `mod = 10` → 4 bytes, `mod = 11` → 0 bytes.

- **Displacement extraction** (`disp_sel.v`): an 8-way mux over bytes B2...B10 selects 32 displacement bits at `disp_pos = opcode_pos + has_sib + two_byte_opcode`, then a final `mux4_32` zero-extends based on `disp_size`.
- **Immediate extraction** (`imm_sel.v`): a 16-way mux over B1...B14 selects 32 bits at `imm_pos = opcode_pos + two_byte_opcode + has_modrm + has_sib + disp_size_dec`; final `mux4_32` zero-extends.
- **Length** (`inst_len.v`): three staged 4-bit adders compute `inst_len = 1 + opcode_pos + two_byte_opcode + has_modrm + has_cs + has_sib + disp_size_dec + imm_size_dec`.

Decode ROM

Per-instruction control signals are produced by 16 64-entry, 32-bit ROM macros (`rom64b32w$`) instantiated in `core/decode/decode_roms.v`. The 64-bit packed `control_signals` bus carries source / destination types, hard-coded register indexes, source-register-index selectors, immediate and operand sizes, `has_cs`, `df_wen` and direction bits, push / pop tags, `addr0 / addr1` load / store tags, branch-class flags (direct, unconditional, NE, NBE), the 4-bit ALU op, the 3-bit MMX ALU op, and the `alu_sel` mux select. The same bus is used by AGEN, LOAD, EXECUTE and WRITEBACK — there is one decode ROM read per instruction.

Branch Prediction

Branch prediction is implemented as a gshare predictor with no separate Branch Target Buffer:

- **Pattern History Table (`pht.v`)**: 16 entries of 2-bit saturating counters, initialised to weakly taken (0b10). Read index = $\text{GHR} \oplus \text{EIP}[3:0]$; the MSB of the indexed counter is the taken/not-taken prediction.
- **Global History Register (`ghr.v`)**: 4 bits. On a conditional direct branch decode, GHR shifts left and inserts the prediction bit. On a flush from EXECUTE the GHR is restored from the corrected_ghr value snapshot at branch time.
- **Targets**: because there is no BTB, the predicted target is computed from the decoded immediate. For near branches, `branch_target_eip = (EIP + inst_len) + sext(imm)`. For far branches with `has_cs = 1`, `branch_target_eip = imm` (absolute).

Branch resolution and PHT update happen in EXECUTE; the corrected GHR replaces speculative shifts on a mispredict.

Next-EIP Logic

`next_eip.v` selects the EIP for the next cycle by a cascaded priority mux:

- 1. **Exception** (highest): `exception_eip` from the EXECUTE-side exception save registers.

- 2. **Mispredict / restateer:** restateer_eip from EXECUTE branch resolution.
- 3. **Branch prediction taken:** branch_target_eip when br_taken is asserted (unconditional direct, or conditional direct with PHT prediction = taken) and Decode is not stalled.
- 4. **Sequential** (default): EIP + inst_len.

When AGEN signals decode_stall, the agen_latch holds its outputs and Decode's eip_reg holds the same EIP so the same instruction is re-decoded the next cycle. GHR updates are gated by !decode_stall so speculative history does not advance under stall. The HLT (0xF4) opcode latches a sticky seen_halt that forces instruction_v = 0 and freezes the EIP until reset.

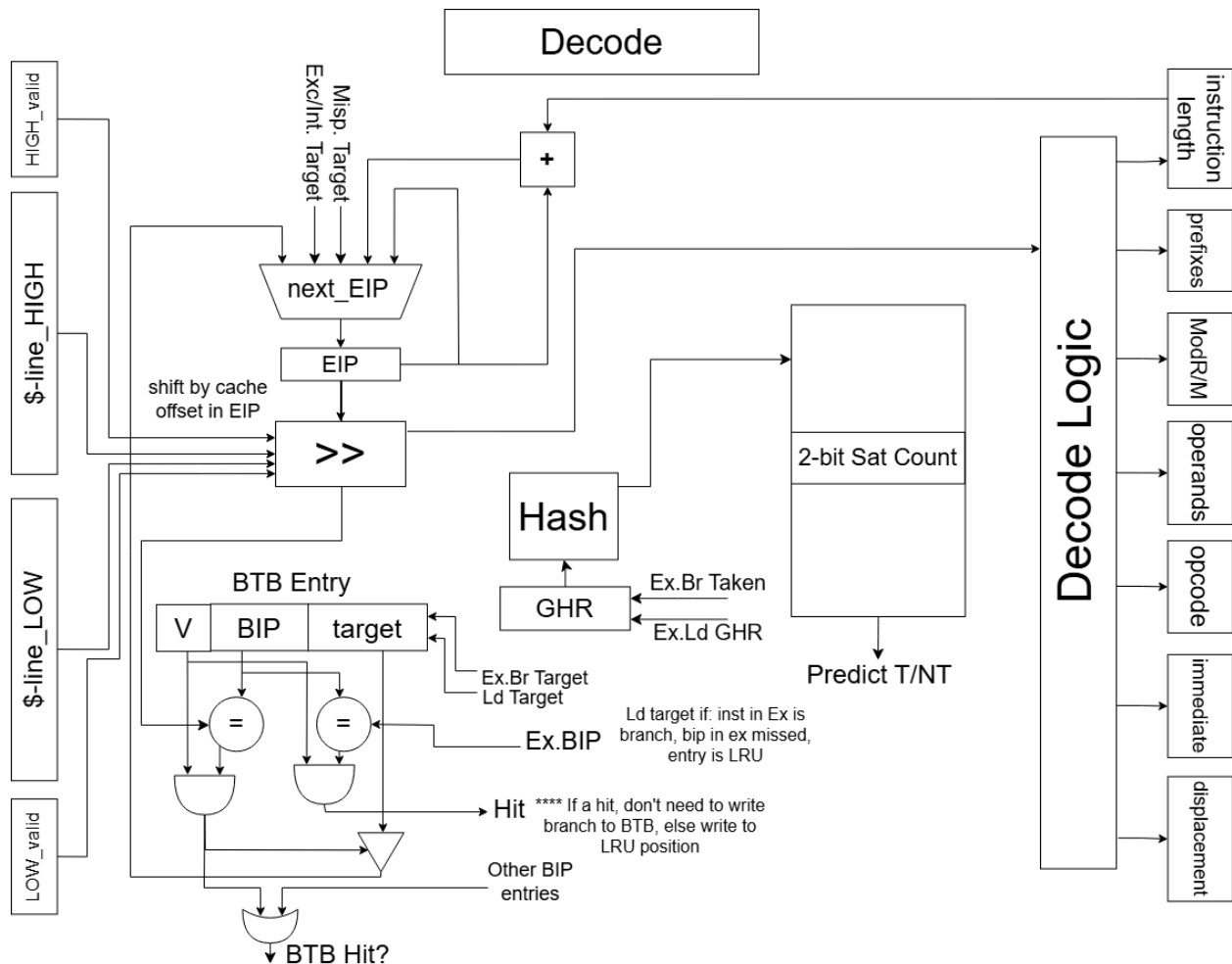


Figure 2. Decode Stage Diagram.

Address Generation

The Address Generation (AGEN) stage reads the register file, computes effective and physical addresses, performs segment-limit checks, drives in-stage architectural updates for STD / CLD / PUSH / POP, and clones REP-string instructions until the iteration count is exhausted. It is also where the speculative-write payload that EXECUTE will later promote into the architectural register file is constructed.

Effective and Physical Addresses

modrm_agu (core/address_generation/modrm_agu.v) implements the standard x86 effective-address formula $\text{effective_address} = \text{base} + (\text{index} \ll \text{scale}) + \text{sext}(\text{disp})$. The base operand is suppressed when the encoding is mod = 00, rm = 101 (raw 32-bit displacement). The index is shifted by 0 / 1 / 2 / 3 according to SIB.scale. The displacement is selected from the 8-, 16-, or 32-bit form and sign-extended. The segmented physical address is formed downstream by adding (segment << 16) to the effective address; the 9-port TLB (ports 7 and 8) performs an early read-only translation for both AGEN address lanes so an MMIO / IO target can be detected before LOAD's seg_limit fault would be latched.

Segment-Limit Checks

seg_limit (core/rf/seg_limit.v) is a five-port combinational comparator that validates $\text{effective_address} + \text{size}$ against a hard-coded per-segment limit (e.g. CS_LIMIT = 0x4FFF). It receives base, size, and seg_idx for each of its ports and produces gp0...gp4 GP-exception flags that ride load_latch into LOAD and EXECUTE. CS / DS / SS / ES / FS / GS each carry their own limit.

REP MOVS / CMPS Cloning

REP-prefixed string instructions are cloned in AGEN: ESI, EDI, and ECX are read through rfp0–rfp2, modified by step amounts produced by modrm_agu, and the new values are written back to the micro-architectural register file via the rep_write_en path. Step direction is controlled by the Direction Flag — STD makes the step a decrement, CLD makes it an increment. The same payload is also latched into load_latch as arf_esi_write_data, arf_edi_write_data, and arf_ecx_write_data so EXECUTE can promote it to the architectural register file when the corresponding micro-instruction retires. AGEN continues to clone until rep_done is asserted by the iteration counter or until an exception fires inside one of the cloned instructions, in which case the cloned EDI / ESI / ECX are written back so REP can resume after the handler returns.

Stack Pointer in AGEN

PUSH and POP perform their ESP update in AGEN via the rep_write_en's sibling stack_write_en path: the new ESP value is written speculatively into rf_gpr, while the pre-decrement value is captured in load_latch.arf_esp_pre_data so a downstream PUSH ESP read in LOAD reads the correct (pre-update) value. EXECUTE then promotes the same payload to the architectural ESP on retirement.

AGEN Stalls

- **Address-source dependency:** rf_dependency_counters reports gpr_src*_ready = 0 when an in-flight write targets a source register used to compute the effective address. AGEN holds and replays.
- **REP register dependency:** REP MOVS / CMPS stalls if any of EDI, ESI, ECX, or DF have an in-flight write (df_dep_nz tracks the DF write counter).
- **REP still cloning:** AGEN issues an internal stall to Decode while it is still emitting cloned copies of a REP instruction.
- **MAR/MDR back-pressure (indirect):** propagated up through LOAD when the bus queues are full.

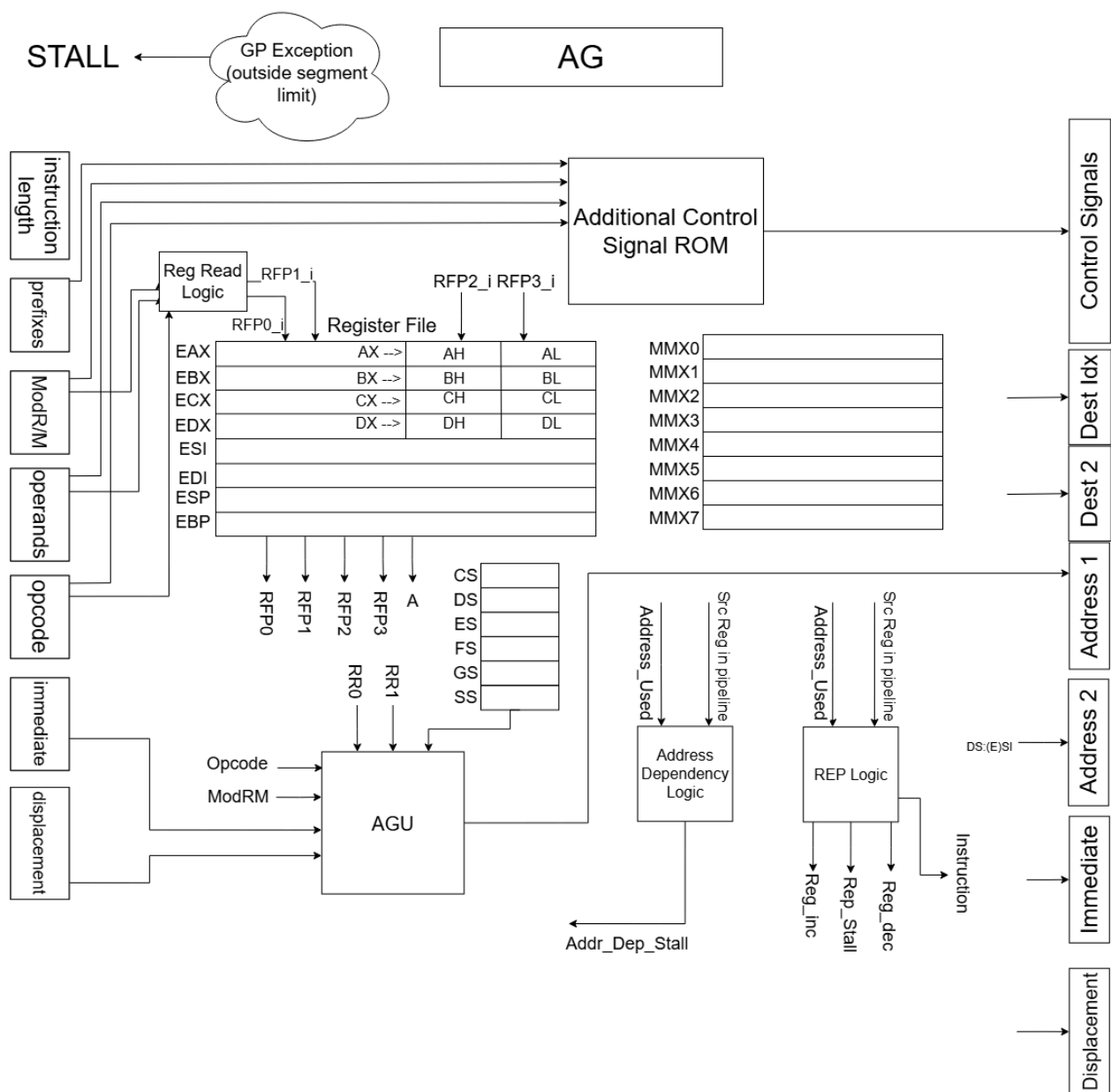


Figure 3. Address Generation Stage Diagram.

Register File

The architectural state lives in four sibling register-file modules — `rf_gpr` (general-purpose), `rf_gpr_arch` (architectural mirror), `rf_seg` (segment), `rf_mmx` (multimedia) — and a dedicated `rf_dependency_counters` block that tracks in-flight writes for stall generation and for flush rollback.

General-Purpose Register File

`rf_gpr` (`core/rf/rf_gpr.v`) holds the eight architectural 32-bit registers (EAX, ECX, EDX, EBX, ESP, EBP, ESI, EDI). It exposes:

- Four read ports — `rfp0` / `rfp1` used by AGEN, `rfp2` / `rfp3` used by LOAD; each port can return an 8-, 16-, or 32-bit slice with the 8-bit form gated to `AH` / `AL` / `BH` / `BL` / `CH` / `CL` / `DH` / `DL` via `index[2]` / `size` logic.
- Two general write ports — `rfp4` / `rfp5` driven by EXECUTE for normal destination writes; collision-with-merge logic at the byte level allows two 8-bit writes that touch opposite halves of the same register to coexist.
- Three special write paths — `rep_write_en` (atomic ESI + EDI step), `stack_write_en` (ESP for PUSH / POP), and `rep_counter_write_en` (ECX decrement). All three are issued from AGEN and re-issued on flush from the architectural register file.
- Five direct outputs — `eax_o`, `ecx_o`, `esp_o`, `esi_o`, `edi_o` — used outside the standard read ports for stack maths, REP cloning, and EFLAGS-related comparisons.

Architectural Register File

`rf_gpr_arch` (`core/rf/rf_gpr_arch.v`) is a structural mirror of `rf_gpr` that only ever sees retired writes — EXECUTE drives `rfp4` / `rfp5` and the speculative-promotion enables (`rep_write_en`, `stack_write_en`, `rep_counter_write_en`) gated on `EX_valid`. Its `arf_ecx_o` / `arf_esp_o` / `arf_esi_o` / `arf_edi_o` outputs are routed through 32-bit muxes back into `rf_gpr`'s special-write ports (PIPELINE.v lines 246–273); on flush, the muxes select the architectural value so a one-cycle pulse rolls ECX / EDI / ESI / ESP back to their last retired state. The OR gates on the corresponding write-enable lines force the special writes high during a flush so the rollback always commits.

Segment and MMX Register Files

`rf_seg` (`core/rf/rf_seg.v`) holds six 16-bit segment selectors (ES, CS, SS, DS, FS, GS). It has two normal read ports plus a dedicated LOAD-stage read port (`seg2_idx_read`) used to read the source segment of a PUSH / MOV without seeing a stale value. Writes go to a single port; CS writes use a separate `cs_write_en` path because they trigger a pipeline flush in EXECUTE.

rf_mmx holds eight 64-bit MMX registers with two 64-bit read ports built from stacked 8:1 muxes and a single 64-bit write port.

Dependency Counters

rf_dependency_counters (core/rf/rf_dependency_counters.v) maintains 6-bit saturating counters per GPR (and per segment register, plus one for DF). A counter increments when AGEN issues a destination index for that register and decrements when EXECUTE retires it; a NOR-tree reduction yields a per-register ready flag. AGEN consumes gpr_src0_ready / gpr_src1_ready / gpr_src2_ready to gate stalls. CS bypasses dependency tracking because a CS write always flushes the pipeline. DF has its own counter (df_inc on STD / CLD launch from AGEN, df_dec on STD / CLD retire from EXECUTE) that gates REP stalls through df_dep_nz.

Load Stage

The Load stage handles all data-side address translation, cache lookups, store-collision checks, and second-half register reads. It also owns the data-cache state machine and the 16-deep instruction queue (IQ) that buffers in-flight memory operations.

Data Cache Architecture

The data cache is a non-blocking, write-back, 4-way set associative cache:

- **Geometry:** 16 sets $\times$ 4 ways $\times$ 8 B/line = 512 bytes total; 9-bit tag (PA[14:7]); 3 bits offset; 4 bits index.
- **Replacement:** absolute LRU per set (lru_store.v) overlaid with per-way unevictable counters that lock a way while a load / store has reserved it.
- **Multi-port write surface:** four “attention” ports gate the data store (data_store.v) — mshr_attention (refill), hsq_attention (store-queue commit), hiq_attention (read-only HIQ probe), evict_attention (read-only eviction read).
- **Tag-store copy:** a parallel tag-store CAM is consulted at IQ enqueue so the hit/miss decision is made before the cycle the line will be needed.

Instruction Queue

All AGEN-issued memory ops drop into the 16-deep IQ (LOAD.v, multi_write_fifo_p with WIDTH = 551). The IQ entry carries everything LOAD and EXECUTE need: vaddr0, vaddr1, vaddr0_ua, vaddr1_ua (unaligned splits), per-byte write masks, GPR / segment / MMX source / destination metadata, op_size, ALU / MMX op, branch metadata, STD / CLD writes, exception flags, and the speculative-write payload (ESI / EDI / ESP / ECX data and enables) destined for the architectural register file. The IQ tail is the LOAD-stage latch; the IQ head (HIQ) is what arbitration grants cache attention.

Unevictable / MSHR Logic

When a load, RWIM, or store enters the IQ, its physical address is checked against the tag-store copy. On a hit the matching way is marked unevictable so it is guaranteed to be present when the instruction reaches HIQ. On a miss a 32-entry MSHR (mshr_dcache.v) is allocated; the MSHR holds {idx[3:0], way[3:0]} and supports K = 3 search ports for HSQ / HIQ / load_latch deduplication — if a request to the same line is already in flight, no new MSHR entry is created and the unevictable counter on the reserved way is bumped instead. If the set is fully unevictable, the IQ stalls.

Cache Arbitration

Three sources contend for the data store every cycle. The arbiter (load_stage_arbiter.v) follows the priority below; thresholds prevent starvation in either direction:

- 1. **Store queue head (HSQ)** — priority 1 when SQ_count $\geq$ SQ_THRESHOLD and the head holds a line.

- 2. **Bus-data response** — priority 2 (and may grant simultaneous EvictionREAD when the target line is dirty and the eviction queue can accept it).
- 3. **Instruction queue head (HIQ)** — priority 3 when $IQ_count \geq THRESHOLD$ and the HIQ address is not aliased by an active MSHR or store-queue entry.
- 4. **HSQ** (regular) when it has a line.
- 5. **HIQ** otherwise (subject to the MSHR / store-queue alias check).
- 6. **Eviction read** when there are pending evictions and the eviction queue is not full.
- 7. Otherwise the cache is idle.

Eviction

Evictions follow these constraints: loads release after HIQ; stores release after HSQ; a store cannot release if another store to the same cache line is in IQ / pipeline / load_latch; a load cannot release if another load to the same line is in IQ / load_latch. dirty_bit_store.v records per-way dirtiness; on a dirty eviction the line is queued in evict_queue (depth 64, sized for a complete halt-time sweep) before being handed to CORE_QUEUES.d_line_q. halt_flush_logic implements the deterministic walk that empties every dirty line on processor halt.

Store-Queue Collision

LOAD checks every entering load against the writeback store queue's dependency CAM (combine_dependent_addr.v, combine_way_check_tags.v). When a load addresses a cache line currently owned by an in-flight store, LOAD waits for the store data to merge in via DATA_MERGER.v rather than re-fetching the line. gen_write_masks.v computes the per-byte write masks from op_size and offset (with a hard-wired 16-bit 0x00FF override for MMX 8-byte stores) and passes them downstream as the byte-level granularity used by both WRITEBACK and the LOAD's collision check.

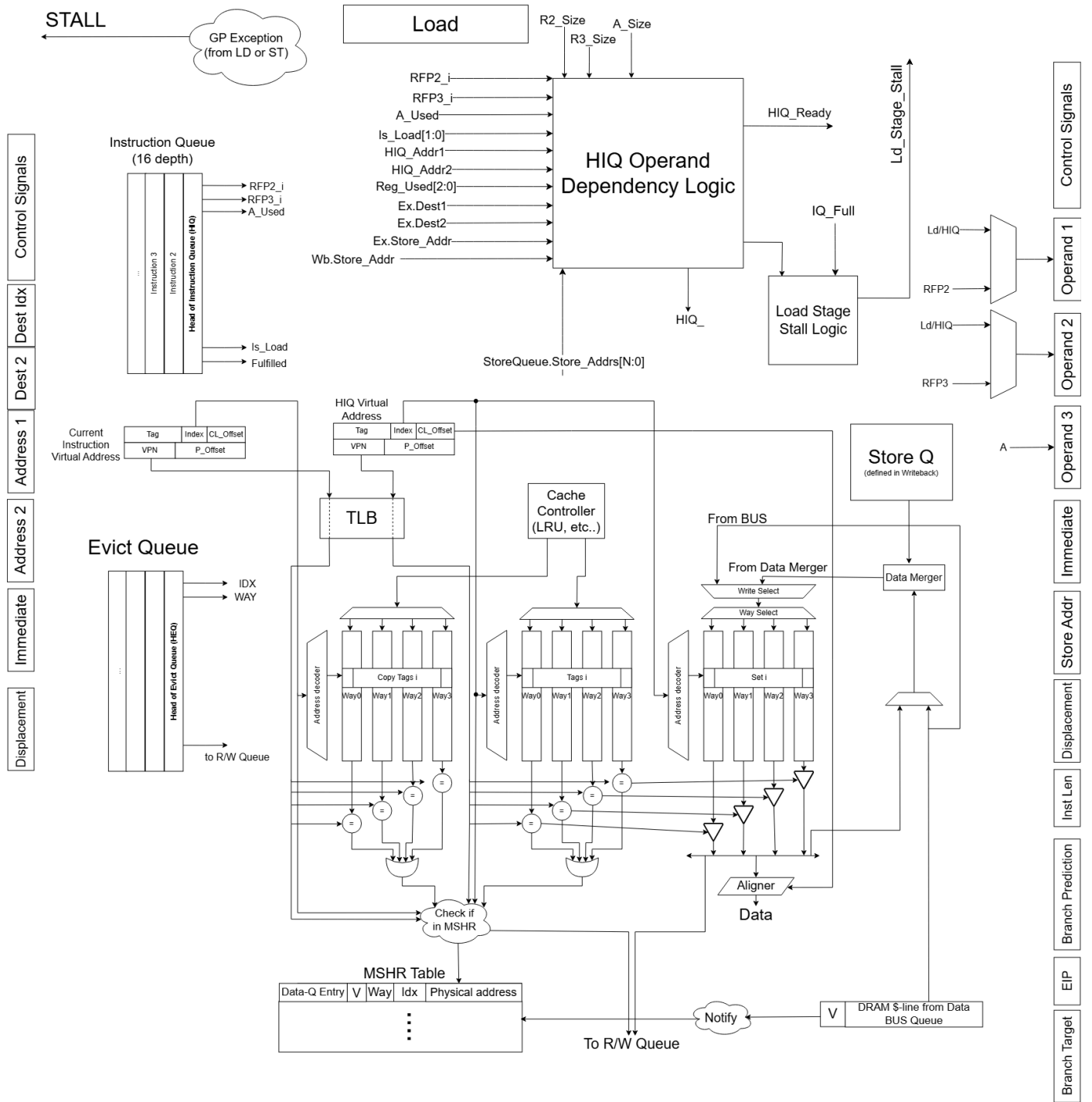


Figure 4. Load Stage Diagram.

Execute

Execute carries out arithmetic, logical, shift, vector, and address-related operations; resolves branches; updates EFLAGS; gates writes to the architectural register file; and detects faults that turn into pipeline-flushing exceptions.

Tiered ALU

alu.v is a 16-lane structural ALU. Operand-size routing on each lane uses width-aware sub-blocks so the same op produces 8-, 16-, and 32-bit results without separate ALU instances:

- **Lane 0** — passthrough for MOV, MOVS, PUSH, POP, CALL, RET, JMP.
- **Lane 1** — BCD / AAA correction (4-bit nibble compared with conditional +6 / +1; updates AF, CF).
- **Lane 2** — BSF; 32-bit priority encoder with index reversal.
- **Lanes 3–5** — ADC, ADD, SBB via adder_p / subtractor_p with carry taps at 8 / 16 / 32 bits.
- **Lane 6** — AND (clears CF and OF).
- **Lane 7** — CMPXCHG; width-aware mask-then-compare with dynamic operand remap.
- **Lanes 9–10** — NOT, OR.
- **Lane 11** — CMPS subtraction (compare result; no data write).
- **Lanes 12–13** — SAL / SAR via parametric barrel_shifter_p with width-aware sign / zero fill and CF tap.
- **Lane 14** — XCHG; swaps src0 / src1 to data0 / data1.

The selection of operand 0 / 1 / 2 / immediate is handled by a series of muxes driven by the decode-ROM bits src*_type and src*_idx_sel; CMPXCHG additionally remaps EAX into src0 and shunts the original src0 to src1 when alu_op = 4'd7.

Specialised Functional Units

- **Barrel shifter** (integrated into ALU lanes 12–13): supports SAL / SAR at 8 / 16 / 32 bits, with the CF tap selected from the width-aware shifter output.
- **MMX / Vector ALU** (mmx_alu.v): 64-bit datapath supporting MOVQ, PADDD (two 32-bit lanes), PADDW (four 16-bit lanes), PAVGB (eight 9-bit zero-extended adders with Cin = 1 then >> 1), PAVGW (four 17-bit), PACKSSWB (16 → 8 with signed saturation via sat_w2b), and PACKSSDW (32 → 16 via sat_d2w). Saturation outputs 0x7F / 0x80 (or 0x7FFF / 0x8000) when the sign-extension check fails.
- **Bit-scan logic**: priority encoder feeding lane 2 for BSF.

- **ASCII / BCD adjust:** dedicated lane (lane 1) handles AAA, with af_new / cf_new conditional on the auxiliary-carry tap.

Branch Resolution

branch_resolution.v computes the actual next EIP (sequential vs predicted-target) and compares it against the prediction latched from Decode. Conditional flag tests (NE = !ZF, NBE = !ZF & !CF) are reduced through OR3 with unconditional_branch to produce true_taken; the mispredict pulse is XOR (prediction, true_taken) AND is_branch. For far branches with has_cs = 1 and direct_branch = 0, target_cs comes from the ALU lane (alu_data1[15:0]); for direct far branches it comes from the decoded immediate. CMPXCHG's conditional swap, REPE CMPS' early-exit pulse, and IRETD's two-μop resteer (latched via iretd_resteer_eip / iretd_resteer_cs DFFs) all merge into the same flush path. PHT and GHR updates are produced from EX_ghr_snapshot ⊕ EX_eip[3:0]; on a branch flush the corrected_ghr replaces the speculative shifts.

Stack and CS Maintenance

CALL, RET, PUSH, and POP make their final stack-pointer changes in AGEN; EXECUTE consumes ESP for the address arithmetic of CALL's return-address store and RET's pop. Direct far branches predict the new CS through Decode (BTB-style entry) and are corrected here on mismatch; indirect far branches are predicted not-taken and corrected here. A non-branch CS write (POP CS, MOV CS) flushes the pipeline and resteeers to the sequential EIP using cs_write_only as the resteer-EIP mux selector.

EFLAGS

Flag generation is split between flag_gen.v (combinational) and eflags.v (registered). flag_gen.v computes ZF, SF, PF, CF, AF, OF, and DF across all operand sizes:

- **ZF / SF:** width-aware NOR / MSB taps on the result; CMPXCHG and CMPS take the subtractor result instead.
- **PF:** even parity over the low byte regardless of op_size (Intel SDM §3.4.3).
- **CF:** width-reconstructed via the $\text{Sum}[n+1] \oplus A[n+1] \oplus B[n+1]$ identity at bits 8 / 16 with direct taps at bit 31; ADD/ADC use the raw cout, SUB/SBB/CMP invert.
- **AF:** bit-4 carry; AAA overrides via bcd_af_cf_tap.
- **OF:** signed-overflow rule at width.
- **DF:** independent of alu_op; CLD / STD set or clear it through df_wen.

eflags.v holds seven DFFs (one per architectural flag) with per-flag write enables. POPF and IRETD-μop-2 bypass flag_gen by selecting the popped flags through a 16-bit mux on the eflags input. If an exception fires during REPE CMPS, the flags revert via the saved exc_flags_saved register populated at the moment exception_flush_v fires.

Exceptions

EXECUTE is the central exception detector. Today, `exception_v = exception_is_gp`; `exception_is_pf` and `interrupt_v` are tied to 0 in EXECUTE.v lines 1285–1286 — page-fault routing and asynchronous interrupts are deferred. GP firing sets `exception_flush_v = exception_v & EX_valid`, which (a) suppresses dest / store / flag writes through the `exception_flush_v_b` gate, (b) latches EIP / CS / EFLAGS / vector into save DFFs, and (c) sets `exception_pending`. The next cycle, `exception_pending` drives flush, retargets the resteer to `{exc_eip_saved[31:4], 4'b0}`, and the FE → DE latch swaps in the hard-coded exception dispatch sequence so Decode begins fetching the handler. `exception_pending` stays high until the routine's closing 0xD4 (AAA) micro-op retires, releasing `exc_routine_active`.

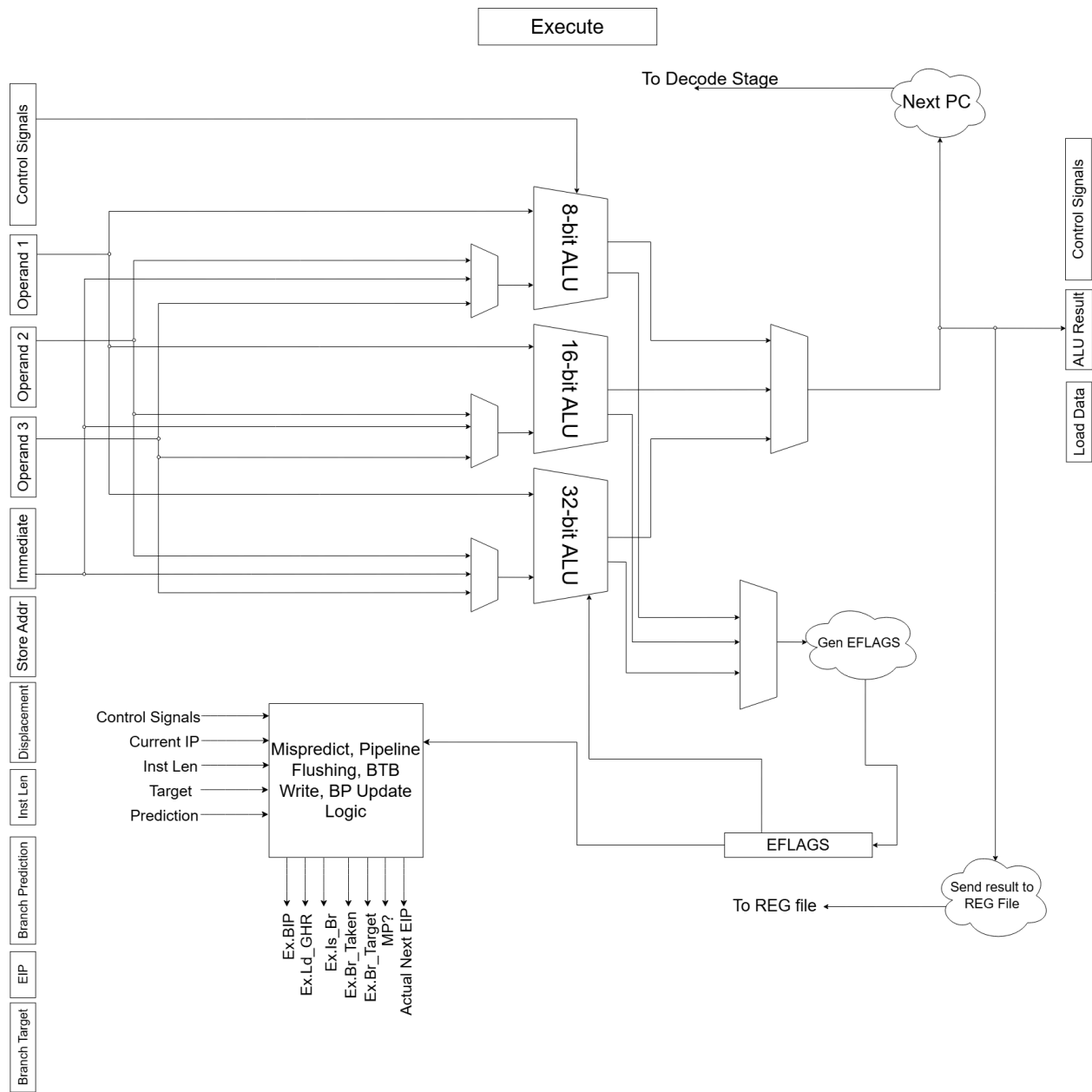


Figure 5. Execute Stage Diagram.

Writeback

Writeback is a queueing stage that commits stores into the data cache without stalling EXECUTE. It is implemented by three CAM-based 32-entry FIFOs that ride alongside the LOAD stage's store-queue:

- **Store-Way Queue (SWQ):** 8 bits per entry, {idx[3:0], way[3:0]}, searchable by the active idx so LOAD can locate the way owning a same-line dependency.
- **Store-Data Queue (SDQ):** 72 bits per entry, {byte_mask[7:0], cacheline[63:0]}, ordered FIFO.
- **Address-Dependency Queue:** 32 bits per entry, {idx[3:0], way[3:0], mask[7:0]}, searched by {HIQ_IDX, HIQ_WAY} so the LOAD-stage collision check uses the exact bytes already computed by AGEN.

Per WRITEBACK cycle there are four ports (lane 0 aligned, lane 0 unaligned, lane 1 aligned, lane 1 unaligned) gated by the per-lane write_mask bits derived in AGEN / WB_latch. The 64-bit operand is left-shifted into a 128-bit window by data0_shamt = {WB_addr0_offset, 3'b0}; the lower 64 bits hit the aligned slot and bits [127:64] cover the unaligned spillover, so a single 8-byte access can straddle a cache-line boundary. Same-cache-line stores merge naturally because the four write_mask bits select which ports actually write at the same SWQ {idx, way}.

Non-stalling commit is achieved by (a) priority promotion of HSQ on the data-cache arbiter when SQ_count crosses the threshold, (b) eviction-aware EvictionREAD grants that overlap with bus-data writes, and (c) per-port write masks so multiple stores to the same line do not serialise.

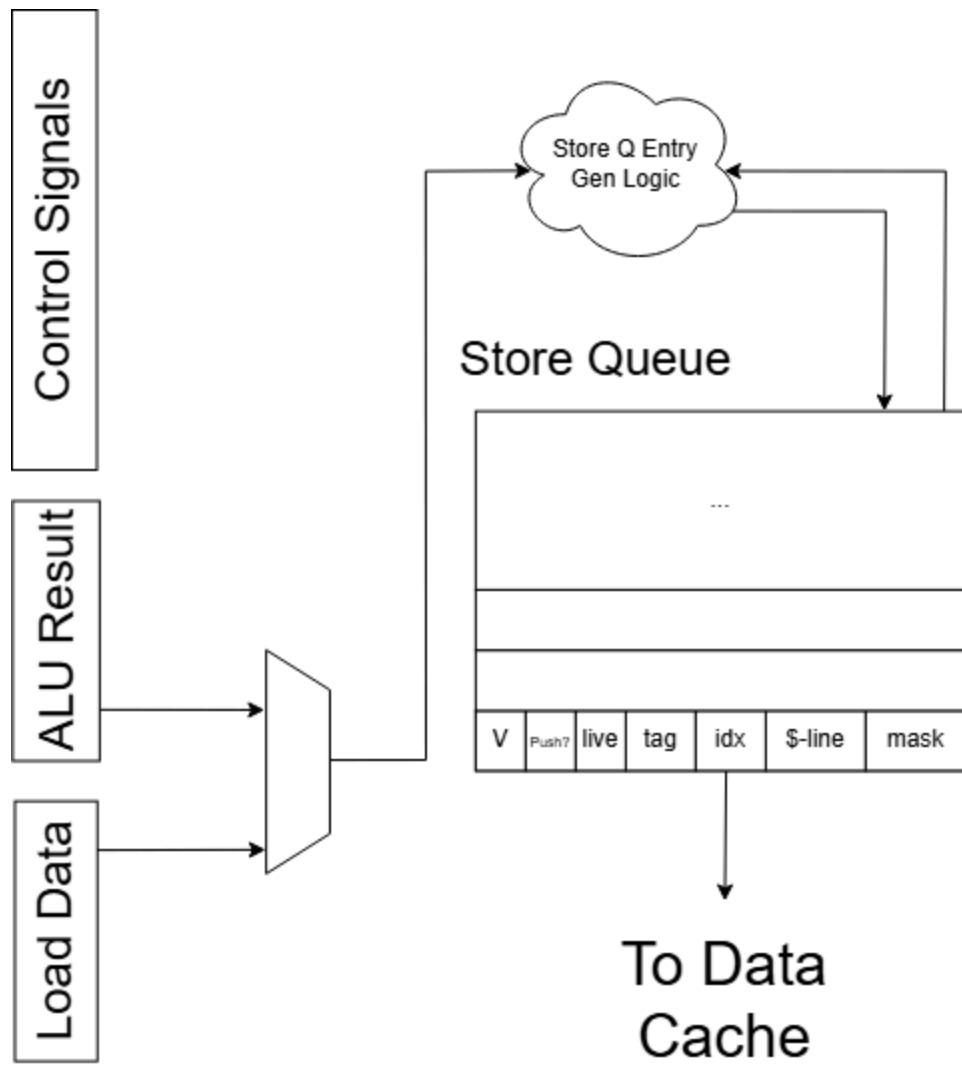


Figure 6. Writeback Stage Diagram.

Main Memory

Main memory is a 32 KB DRAM array organised as 4 banks of SRAM-style cells with a 32-bit row buffer per bank. The array, the DRAM controller, and the MAR / MDR queues all live in OFF_CORE.

DRAM Array

- **Capacity:** 32 KB total = 4 banks $\times$ 16 tiles/bank $\times$ 128 rows/tile $\times$ 4 B/row.
- **Bank pairing:** banks 0 / 1 form one pair, 2 / 3 form the other. `paddr[2]` selects the pair, `paddr[1]` selects the bank within a pair.
- **Address bits to DRAM (11 bits):** [10:7] tile, [6:0] row.
- **Row buffer:** 32 bits per bank $\rightarrow$ 128 bits aggregated across the four banks; `LD_EN` latches the row buffer.
- **Cache-line layout:** an 8-byte D-cache line lives in two beats from a single bank pair; a 16-byte I-cache line lives in four beats from the four banks.

MAR Queue

`mar_q` (`off_core/queues/mar_q`) is 16 deep and 16 bits wide per entry: bit 15 is `dma_tag`, bit 14 is `i / d`, and bits [13:0] hold the `{r/w, paddr[12:2]}` pruned address. The queue is pushed by `arb_write_mar` (delayed one cycle to align with the arbiter grant), and popped on `stream_done` (read complete) or `pl_dn` (write pulse complete). The `dma_tag` is latched per entry so the DRAM controller can switch into the 4-byte DMA write path even after the queue has depth.

MDR Queue

`mdr_q` (`off_core/queues/mdr_q.v`) is 8 deep and 64 bits wide per entry. It assembles two 32-bit beats from the data bus into one 64-bit DRAM-write entry: the first beat lands at `bit_ptr = 0`, the second at `bit_ptr = 1`, and `burst_complete` advances the tail. A dual full-flag (exact + almost-full) prevents push when the last in-flight beat is still in flight. The MDR's write enable is the OR of `d_line_bus_grant_dram` and `dma_bus_grant_r` so both eviction and DMA paths share the same 2-beat assembler.

DRAM Controller

`dram_ctrl` (`off_core/dram_ctrl/dram_ctrl.v`) is a 7-state FSM driving the per-bank `CE / OE / WR / LD_EN` signals and a one-hot `stream_sel` that walks the row-buffer chunks onto the data bus:

- **State 0 – IDLE:** waits for `MAR_Q` non-empty; latches address and `bank_mask`.

- **State 1 – RD_READ:** asserts CE / OE for $N_READ = \text{ceil}(75 \text{ ns} / \text{BUS_CLK_PERIOD})$ cycles.
- **State 2 – RD_LATCH:** captures row buffer; arms the streamer.
- **State 3 – STREAM:** rb_streamer drives one 32-bit chunk per cycle until stream_done. Token order is bank-mask-driven so an I-cache miss streams 4 chunks (16 B) and a D-cache miss streams 2 chunks (8 B).
- **State 4 – WR_SETUP:** asserts setup signals for $N_SETUP = \text{ceil}(25 \text{ ns} / \text{BUS_CLK_PERIOD})$ cycles.
- **State 5 – WR_PULSE:** asserts CE / WR for $N_PULSE = \text{ceil}(25 \text{ ns} / \text{BUS_CLK_PERIOD})$ cycles.
- **State 6 – WR_HZ:** holds outputs Hi-Z for $N_HZ = \text{ceil}(18 \text{ ns} / \text{BUS_CLK_PERIOD})$ cycles before returning to IDLE.

Active-low CE / OE / WR are masked per bank via bank_mask AND/NAND so only the banks selected by the address pair are driven. LD_EN is active-high. stream_sel is generated by AND-ing the active flag from State 2 with a token mask from stream_counter, which shifts left through the asserted banks until reaching the highest-asserted bank.

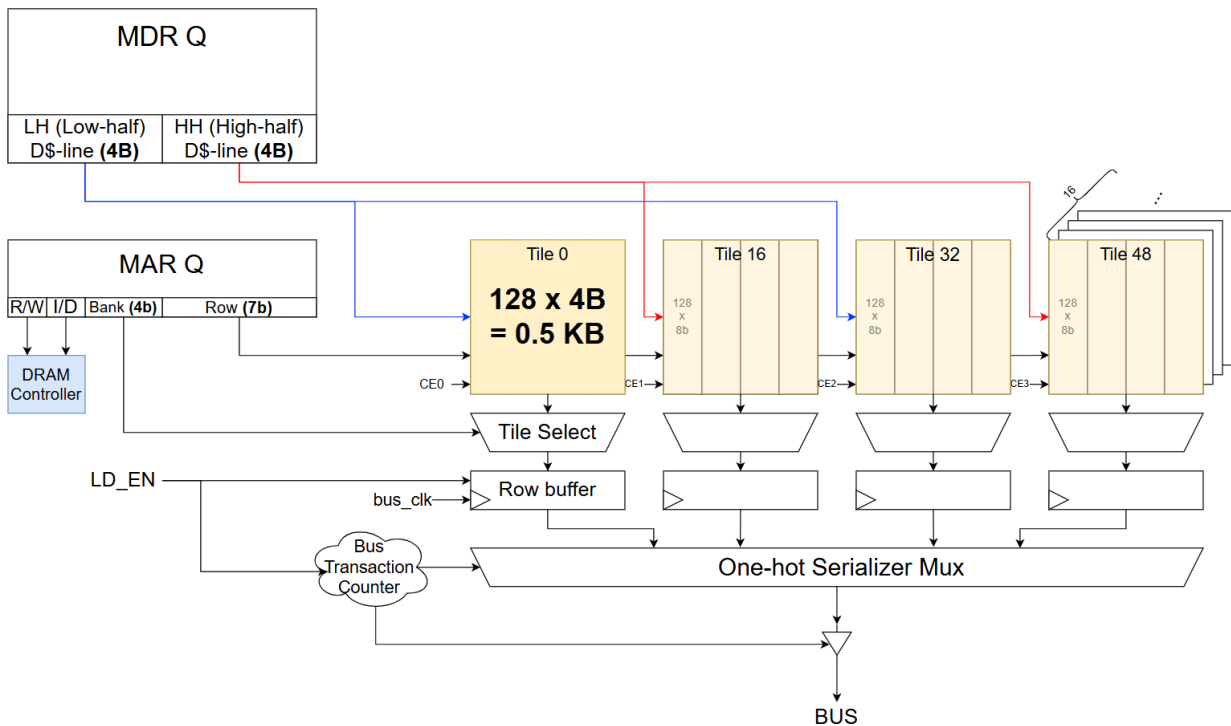


Figure 7. Main Memory Diagram.

Interrupts and Exceptions

The interrupt / exception subsystem detects, prioritises, drains, and dispatches synchronous exceptions. In the current implementation the fault sources actually wired into EXECUTE are limited to General-Protection faults driven by AGEN's `seg_limit` and TLB violations carried in `vaddr0_gp / vaddr1_gp`; the page-fault and asynchronous-interrupt inputs are reserved (tied to 0 in EXECUTE.v lines 1285–1286) so the routing for IDT-based handlers is in place but not yet exercised.

Detection in EXECUTE

Once per cycle EXECUTE OR-reduces `exception_is_gp`, `exception_is_pf`, and `interrupt_v` into `exception_v`. `exception_flush_v = exception_v & EX_valid`, gated combinationally and registered, suppresses every retirement effect of the offending micro-op:

- Destination write enables (`dest0_v_valid`, `dest1_v_valid`) are AND-gated with `exception_flush_v_b`.
- Store-queue allocation (`is_store_in`) is AND-gated with `exception_flush_v_b`.
- EFLAGS write enable (`flag_wen_eff`) is AND-gated with `exception_flush_v_b`.
- Speculative architectural-RF promotion (`rep_write_en`, `stack_write_en`, `rep_counter_write_en`) is gated indirectly through `EX_valid` — because `EX_valid` drops, no aRF write fires.

Pass / Reg Save

Four save registers latch the architectural context at the moment `exception_flush_v` fires:

- **exc_eip_saved (32 b)** — the offending instruction's EIP, pushed back as the resteer EIP one cycle later.
- **exc_cs_saved (16 b)** — the live CS at fault time.
- **exc_flags_saved (16 b)** — the architectural EFLAGS at fault time, restorable by IRETD.
- **exc_vector_saved (8 b)** — the handler vector. Today this is hard-wired to `gp_vector = 0x58` (`true_exception_vector`). The infrastructure for prioritising multiple sources by age is present (the save mux only updates on the cycle the exception is granted) but a full priority encoder across pipeline stages is deferred.

Drain and Dispatch

`exception_pending` is set the cycle after `exception_flush_v` and stays high until the closing 0xD4 (AAA) micro-op of the exception routine retires. This keeps a single in-flight exception sticky through the resteer cycle and through the routine's prologue. Resteer logic in EXECUTE selects the `exception_target_eip = {exc_eip_saved[31:4], 4'b0}` over the IRETD / branch resteer paths whenever `exception_pending` is asserted.

On the same edge, FETCH's `decode_latch` swaps in a hard-coded exception dispatch pattern generated on the fly from the IDTR base (0x02000010) and the faulting EIP (`decode_latch.v` lines 238–250) — a 13-byte software exception sequence that pushes EFLAGS / CS / EIP, computes the IDT entry address as $\text{IDTR.Base} + (\text{Vector} \times 8)$, reads the new CS:EIP, and resumes execution. Because the routine is materialised as ordinary instructions in the pipeline, no separate microsequencer FSM is required.

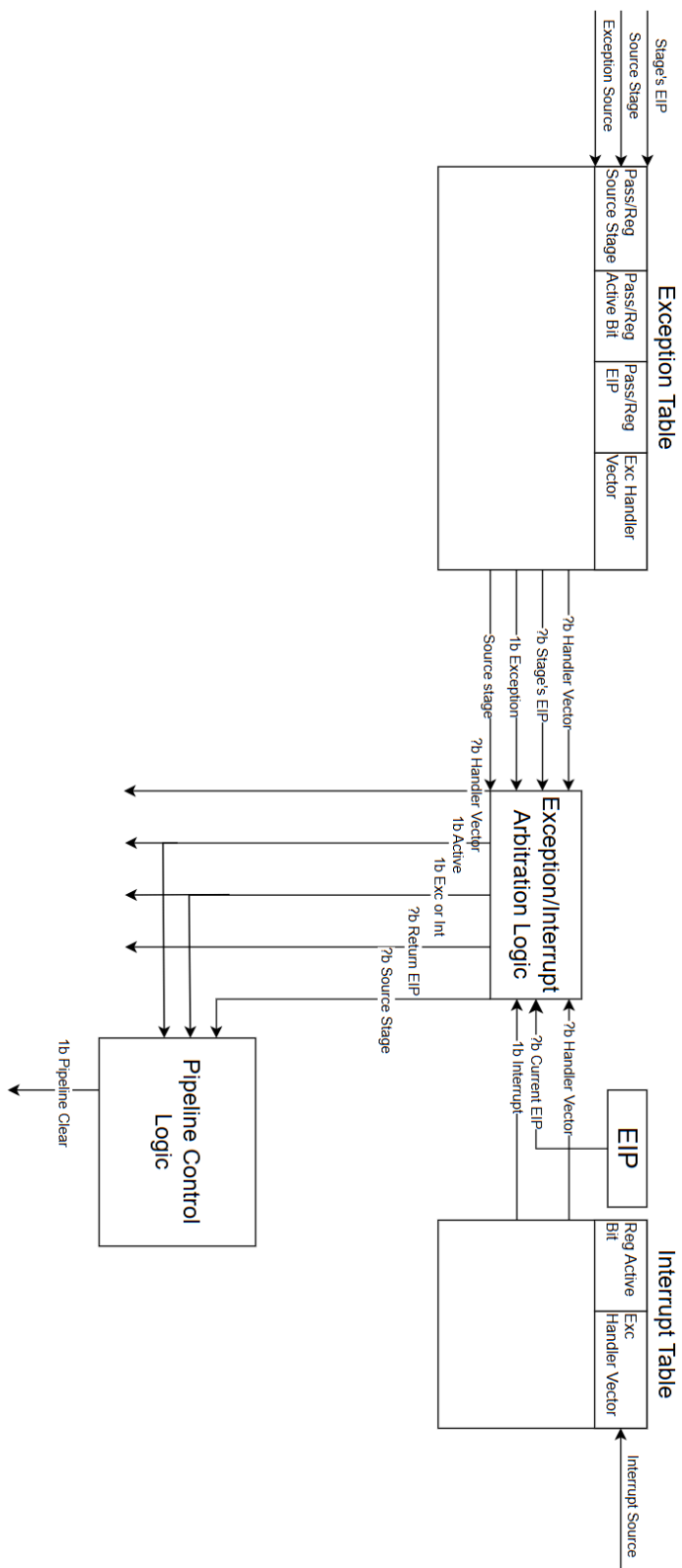


Figure 8. Interrupt and Exception Handling Logic.

Bus Interface

All CPU/DRAM traffic is multiplexed across two tri-state buses: a 16-bit address bus and a 32-bit data bus. The CPU side of those buses is owned by CORE_QUEUES (core/bus_queues/CORE_QUEUES.v); the OFF_CORE side is owned by mar_q, mdr_q, DRAM controller, DRAM, DMA (Direct Memory Access) controller, and keyboard I/O.

CPU-Side Queues (CORE_QUEUES)

- **i_rw_q** — 16 deep, 16 bits per entry {r/w, pa[14:0]}. Holds outstanding I-cache miss addresses. Drives the address bus when granted.
- **i_bus_q** — 3 deep, 128 bits per entry. Assembles four 32-bit beats from a DRAM read into one 128-bit I-cache line and hands it to the I-cache.
- **d_rw_q** — 16 deep, 17 bits per entry: {is_io, r/w, pa[14:0]} for two D-cache lanes.
- **d_line_q** — 4 deep, 64 bits per entry. Buffers dirty cache-line write-backs and IO-store payloads. Per entry an is_io tag selects 1-beat IO writes vs 2-beat eviction bursts.
- **d_bus_q** — 3 deep, 64 bits per entry. Assembles two 32-bit beats from a DRAM data read into a 64-bit D-cache line.
- **io_data_queue** — 4 deep, 32 bits per entry. Captures single-beat I/O responses (e.g. KBSR / KBDR).

Address-Bus Arbitration

bus_arbiter.v's address-bus arbiter has a single fixed priority order: i_rw_q (highest) > d_rw_q > DMA. Whichever queue is granted drives the address bus through a tri-state. arb_write_mar pushes a corresponding entry into MAR_Q; for IO writes the MAR_Q push is suppressed because IO traffic does not touch DRAM. IO writes additionally synchronise with the d_line_q (which holds the IO payload) to ensure the address and data grants align.

Data-Bus Arbitration

The data-bus arbiter has four contenders, with the priority below. Burst locking (burst_lock_dline, burst_lock_dma) keeps a 2-beat write-back atomic so that MDR_Q's 2-beat assembler is never starved mid-burst:

- **1. Keyboard I/O response** (highest) — highest because IO reads are short and rare; blocked only by an active burst lock or by an in-progress DRAM drive.
- **2. DRAM controller return** (read response).
- **3. d_line_q write-back** (CPU eviction or IO-store).

- 4. **DMA payload** (lowest) — deliberately lowest so it never starves CPU traffic.

DMA Controller and Disk Model

The DMA controller (`off_core/dma/dma.v`) is exposed to the CPU as four 32-bit MMIO registers and acts as a bus master for transfers from the disk-side buffer into DRAM:

- **0xFA10 src_addr** — disk source address.
- **0xFA14 dst_addr** — physical-memory destination address.
- **0xFA18 size** — transfer size in bytes (low 13 bits, ≤ 4 KB).
- **0xFA1C go** — trigger bit; writing 1 starts a transfer.

A 4 KB internal buffer (`disk_buffer` reg [32767:0]) is filled by a behavioural disk model (`off_core/disk.v`) that imposes a 750 ns `FILL_NS` latency and a byte-addressable backing memory the testbench preloads. Once the `go` signal fires, the FSM walks `IDLE` → `FILL` → `ADDR` → `LSB` → `MSB` (→ `WAIT` if `d_line_q` is contending) → `DONE`. `ADDR` drives `dma_addr_out = dst_addr + byte_offset`; `LSB` / `MSB` drive the 32-bit beats. The `MAR` entry carries `dma_tag = 1` so the DRAM controller switches into the 4-byte bank-mask write path; `OFF_CORE.arb_dma_req` is intentionally tied to 0 in `TOP` so the 8-byte write path is used and `MDR`'s 2-beat assembly is preserved (see `TOP.v` header comment for the rationale).

Keyboard MMIO

`keyboard_io` (`off_core/io/keyboard_io.v`) implements two memory-mapped 16-bit registers:

- **KBSR (0xFA00)** — **status**. Bit 15 is the ready flag. It is set on a detected change in `kbd_in` (edge-detected by `detect_change_p`) and cleared on the cycle `KBDR` is granted the data bus.
- **KBDR (0xFA02)** — **data**. Bits [15:0] hold the most recently received character.

On a CPU MMIO read the address is decoded by `io_addr_hit`; a pending request is latched into `pending_q` and `kbd_io_data_bus_req` is asserted to the bus arbiter. When `arb_data_bus_grant_io` fires, a tri-state driver drives `KBSR_word = {16-bit zeros, kbsr_q, 15-bit zeros}` or `KBDR_word = {16-bit zeros, kbdr_q}` on the data bus and (for `KBDR`) clears the ready bit.

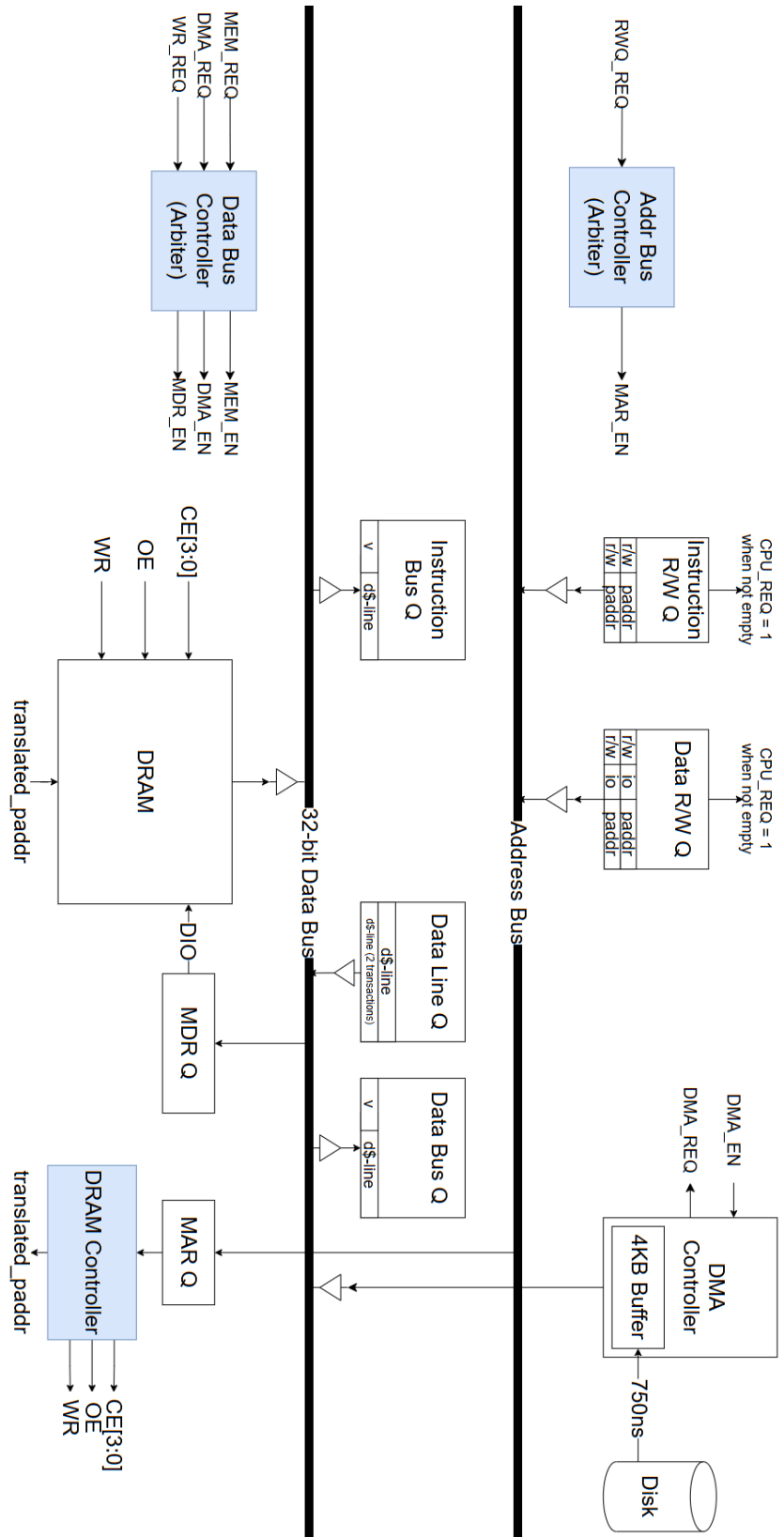
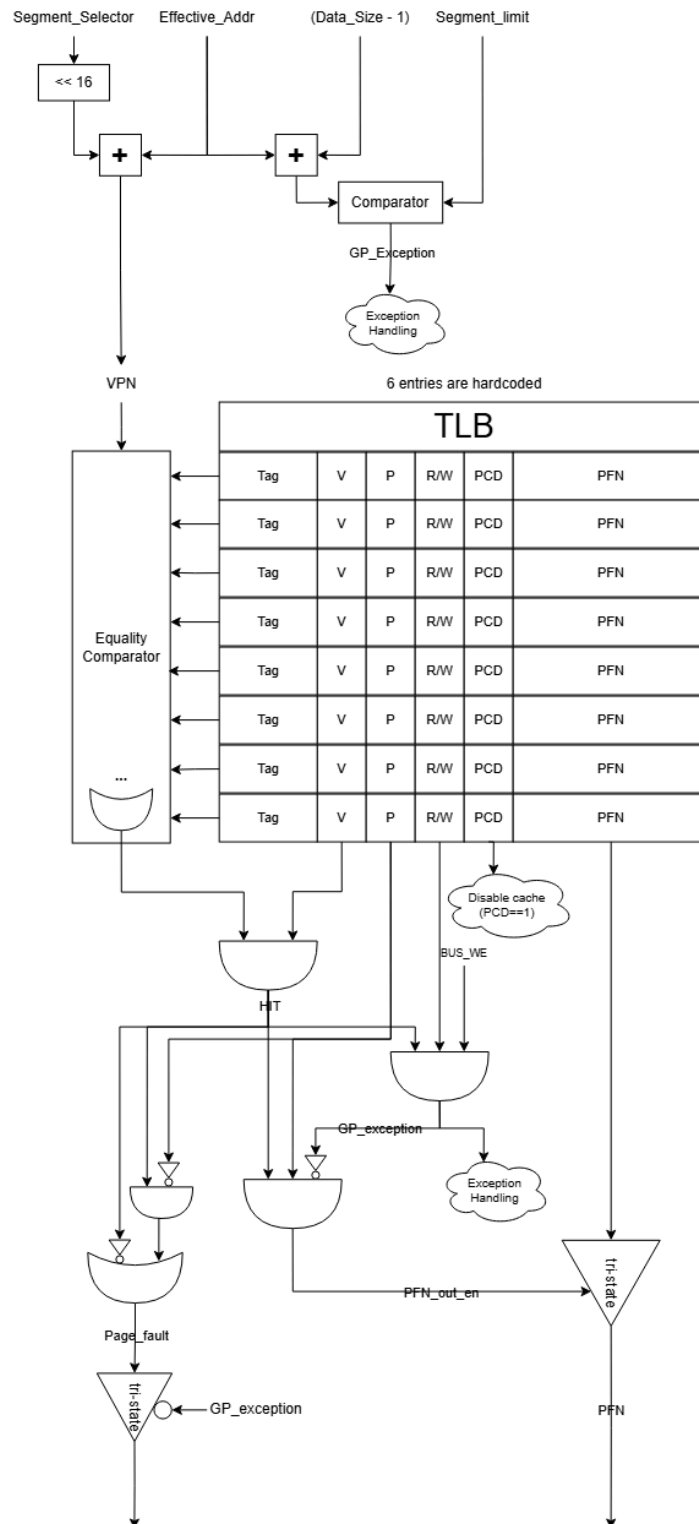


Figure 9. Bus Interface.

Appendix A

TLB Microarchitecture



Appendix B

Populating Instruction Queue Logic

if(IQ = full):

1. STALL

Else:

1. Check Hit/Miss using Tag Store Copy

if(miss):

1. Place in MSHR
2. Choose LRU way to evict
3. Mark way as invalid

Appendix C

Data Cache Arbiter Logic

```
if(SQ_count >= SQ_THRESHOLD and HSQ has $-line)
    1. Grant HSQ
Else if(BusData_valid AND target $line is dirty AND DataLineQ_not_full):
    1. Grant EvictionREAD and Grant BUSQ Write
Else if(BusData_valid):
    1. Grant BUSQ Write
Else if(IQ_count >= THRESHOLD && HIQ_Addres not in MSHR && HIQ_Address not in STOREQ):
    1. Grant HIQ
Else if(HSQ has $line):
    1. Grant HSQ
Else if(IQ != empty && HIQ_Addres not in MSHR && HIQ_Address not in STOREQ):
    1. Grant HIQ
Else if(pending evictions && DataLineQ_not_full):
    1. Grant EvictionREAD
Else:
    1. The $ is idle!
```

Appendix D

ALU Function Breakdown

- Adder/Subtractor
 - ADC, ADD, SBB
- Barrel Shifter
 - SAL, SAR
- Compare
 - CMPXCHG, REPE CMPS
- Branch Target Adder
 - CALL, JMP, JNBE, JNE
- Branch Condition Evaluator
 - CMOVC, JNBE, JNE
- Condition Flags Generator
 - AAA, ADC, ADD, AND, BSF, CLD, CMPXCHG, OR, REPE CMPS, SAL, SAR, SBB, STD
- Bit Scan Logic (priority encoder)
 - BSF
- Vector ALU
 - PADDW, PADDD, PAVGB, PAVGW
- SIMD Packing Unit
 - PACKSSWB, PACKSSDW
- Stack Pointer Update
 - CALL, IREtd, POP, PUSH, RET
- Data Routing Muxes
 - CMOVC, MOV, MOVQ, MOVS, XCHG
- Bitwise Logic Unit
 - AND, OR, NOT
- ASCII Adjust Logic
 - AAA

Appendix E

Address Bus Arbitration Logic in Conditional Statements

```
if (MAR_Q full):  
    NOP  
else if (I_RW_Q != empty):  
    I_RW_Q_GRANT = 1           // instruction fetch  
else if (D_RW_Q != empty && (~IO-write || D_line_Q IO-data ready)):  
    D_RW_Q_GRANT = 1           // data R/W (IO writes wait for matching data)  
else if (DMA_ADDR_REQ):  
    DMA_ADDR_GRANT = 1  
else:  
    NOP
```

Appendix F

Data Bus Arbitration Logic in Conditional Statements

```
if (burst_lock_dline && D_line_Q != empty):
    D_LINE_Q_GRANT = 1      // 2nd beat of cache-line writeback burst
else if (burst_lock_dma && DMA_DATA_REQ):
    DMA_DATA_GRANT = 1      // 2nd beat of DMA burst
else if (kbd_io pending):
    IO_GRANT = 1            // I/O read response (blocks dram_ctrl too)
else if (DRAM_CTRL data ready && BUS_Q != full):
    DRAM_CTRL_GRANT = 1     // DRAM return
else if (D_line_Q != empty && MDR_Q != full && IO-sync ok):
    D_LINE_Q_GRANT = 1      // cache-line writeback (1st beat)
else if (DMA_DATA_REQ && MDR_Q != full && D_line_Q empty):
    DMA_DATA_GRANT = 1      // DMA data (1st beat)
else:
    NOP
```